

GRAPHWORD

Arquitectura distribuida, demostración en AWS, pruebas CI/CD y benchmark

7 de octubre de 2026

ULPGC

Guillermo Cubas Granado

Resumen

GraphWord construye y consulta grafos de palabras. Cada palabra constituye un vértice y dos palabras se conectan cuando tienen la misma longitud y difieren en una única letra. El proyecto desarrolla este modelo desde un motor Python por consola hasta una aplicación con API HTTP, persistencia, trabajos asíncronos y construcción repartida entre workers en AWS.

La solución separa FastAPI, el motor, la coordinación y los adaptadores de almacenamiento. En local utiliza SQLite; en AWS emplea S3 para objetos, DynamoDB para estados y SQS para avisos. La entrada pública HTTPS se implementa mediante Lambda y Mangum. Los cálculos distribuidos se ejecutan en dos EC2 y se coordinan con reservas temporales, heartbeat, reintentos y un reconciliador.

Las evidencias conservadas incluyen 122 pruebas ejecutadas en Python 3.11 y 3.12, una demostración HTTPS y otra de construcción distribuida en EC2. La revisión local del 7 de octubre reproduce con éxito los 122 casos de la aplicación. El benchmark AWS registra 24 construcciones correctas y aceleraciones de aproximadamente 1,39 a 1,46 veces con dos workers frente a uno. Los resultados corresponden a cargas pequeñas y a un protocolo concreto.

El informe explica modelo, algoritmos, comunicaciones, arquitectura, demostración, pruebas, automatización y rendimiento. Distingue propiedades del código, resultados locales y registros históricos. Sus conclusiones identifican límites de memoria, coordinación, seguridad y observabilidad.

Palabras clave: grafos de palabras, FastAPI, AWS, procesamiento distribuido, S3, DynamoDB, SQS, CI/CD, benchmark.

Índice

Resumen.....	2
1 Introducción y objetivos	4
2 Metodología y evidencias	4
3 Modelo de grafo y corpus	5
4 Construcción y algoritmos	7
5 Arquitectura y comunicaciones de la API.....	8
6 Contrato de la API.....	9
7 Arquitectura AWS.....	11
8 Coordinación y recuperación	12
9 Seguridad y operación.....	13
10 Demostración funcional	14
11 Pruebas y validación	17
12 Integración y despliegue continuos	18
13 Diseño experimental.....	19
14 Benchmark y discusión.....	20
15 Limitaciones y mejoras	21
16 Conclusiones	23
Referencias	24

1 Introducción y objetivos

1.1 Problema planteado

Un grafo de palabras permite estudiar transformaciones ortográficas de una letra. La secuencia cat, bat, bad, dad representa tres cambios válidos si esas palabras pertenecen al diccionario. Una transformación depende tanto de la relación de adyacencia como del conjunto de palabras admitidas.

El problema combina construcción y consulta correctas con ingeniería de servicios. Al ofrecerlo mediante una API, hay que validar entradas, conservar resultados y coordinar procesos que pueden fallar o competir por trabajos. GraphWord permite estudiar conjuntamente algoritmos, persistencia y procesamiento distribuido.

1.2 Objetivos

El objetivo general es implementar y evaluar una aplicación que mantenga la corrección al pasar de ejecución local a arquitectura distribuida en AWS.

Los objetivos específicos son:

- Definir una relación precisa y un motor independiente de servicios cloud.
- Repartir la construcción sin perder conexiones y contrastarla con un oráculo independiente.
- Exponer construcciones y consultas mediante HTTP con validación.
- Persistir resultados y estados, recuperar interrupciones y tolerar avisos repetidos.
- Automatizar pruebas y despliegue con identificación de revisión.
- Comparar uno y dos workers y discutir las condiciones de medición.

1.3 Alcance

La solución aborda grafos no dirigidos y sin pesos, con palabras ASCII normalizadas a minúsculas. Incluye caminos mínimos, búsquedas acotadas de caminos simples, grados, componentes y regiones k-core. El corpus curado reúne palabras inglesas de tres a ocho letras.

Las consultas se ejecutan en el proceso API. La distribución entre máquinas se aplica a la construcción mediante partición y reducción. La relación entre palabras es ortográfica: el código no incorpora embeddings, modelos de lenguaje ni llamadas a una API de inteligencia artificial. [C1, C2]

2 Metodología y evidencias

2.1 Procedimiento

La evaluación combina inspección del código y configuración, comprobación de datos, pruebas locales y análisis de registros. La versión de trabajo examinada corresponde a HEAD face65a; GraphWord-CD, a 985f2c0. Los registros históricos identifican otras revisiones y se presentan por separado.

El oráculo del constructor examina parejas de palabras sin repetir la estrategia de patrones. Las medianas y aceleraciones se recalculan desde los JSON originales, conservando unidades, repeticiones y componentes incluidos.

Evidencia	Qué permite sostener	Límite
Código y plantillas	Funciones y recursos implementados	No acredita un servicio activo
Pruebas locales	Casos ejecutados correctamente	No reproduce todos los fallos cloud
Registros CI	Resultado por revisión e intérprete	No indica cobertura completa
Smoke AWS	Operaciones de una ejecución concreta	No acredita disponibilidad posterior
Benchmark	Tiempos bajo su protocolo	No representa otras cargas

Las comprobaciones nuevas son locales. Las evidencias AWS son históricas: no se ha efectuado un nuevo despliegue o benchmark externo. Los JSON de benchmark no guardan fecha y commit, por lo que no se les asignan datos de ejecución que no contienen. [E1-E6]

2.2 Organización

Directorio	Función
GraphWord-python	Motor inicial y demostración por consola
GraphWord-work	Aplicación con API persistencia y ejecución local o AWS
GraphWord-CD	Controlador de despliegue del laboratorio
informe	Documentación documentos y evidencias
respaldo-github	Copia de seguridad Git previa a una limpieza

GraphWord-python utiliza biblioteca estándar y ejecuta secuencialmente las particiones de la CLI. GraphWord-work amplía el motor e incorpora servicios. El parche y el bundle son conservación, no componentes de ejecución.

Parte de la documentación describe propuestas como ECS, ALB o ECR y pendientes anteriores de CI/CD. Se contrasta con el código para evitar presentarlas como recursos implementados. No se evalúa el código Java original mencionado en los antecedentes. [C1, C7]

3 Modelo de grafo y corpus

3.1 Vértices aristas y representación

Los vértices son las palabras normalizadas. Una arista une dos palabras distintas cuando tienen igual longitud y una única posición diferente. La relación es simétrica y cada arista representa un cambio: el grafo es simple, no dirigido y sin pesos.

Un diccionario Python asocia cada palabra a un conjunto de vecinos. Los conjuntos evitan duplicar conexiones. La memoria de adyacencia crece como $O(V+E)$, con V vértices y E aristas, además del texto y sobrecoste de objetos Python. [C2]

3.2 Normalización

El motor elimina espacios extremos, transforma a minúsculas, acepta letras de a a z, elimina duplicados y ordena. No comprueba significado: una cadena puede ser sintácticamente válida sin ser una palabra.

El intervalo de tres a ocho letras pertenece al corpus curado y no al motor general. La API admite listas del usuario, mientras el catálogo ofrece conjuntos documentados.

3.3 Fuentes y transformación

El corpus combina el diccionario estadounidense de LibreOffice, derivado de Hunspell y SCOWL, con CMUdict. Las revisiones de origen se fijan en el manifiesto y se conserva la intersección de formas admitidas. [C3]

De Hunspell se leen formas base, se retiran marcas de afijos y se excluyen entradas NOSUGGEST, sin expandir todas las flexiones. De CMUdict se unifican variantes y se exigen fonemas conocidos con al menos una vocal marcada 0, 1 o 2. La marca 0 se acepta y representa ausencia de acento.

Se conservan formas originalmente minúsculas ASCII de tres a ocho letras, se ordena y se separa por longitud. La lista léxica intermedia contiene 22.070 formas y la de pronunciación 84.373. Su intersección reúne 16.491 palabras. Los ocho outputs, incluidas ambas listas intermedias, coinciden localmente con cantidades y SHA-256 del manifiesto.

3.4 Resultados estructurales

Diccionario	Vértices	Aristas	Componentes	Aislados	Grado máximo
words3	628	3.815	12	8	26
words4	1.889	7.153	117	89	25
words5	2.677	3.171	823	640	15
words6	3.676	2.332	2.164	1.766	15
words7	3.898	1.187	3.053	2.677	11
words8	3.723	432	3.327	3.033	5

Estas estadísticas se calculan localmente con el motor actual y no son tiempos de benchmark. En este corpus, las palabras largas forman grafos más fragmentados. Más vértices no implican más aristas: words4 tiene más conexiones que words6.

La combinación reúne 16.491 vértices, 18.090 aristas, 9.496 componentes y 8.213 aislados. No hay aristas entre longitudes. La tendencia depende del corpus y no constituye una ley general del idioma.

3.5 Integridad licencias y límites

La carga S3 verifica UTF-8, tamaño, orden, duplicados, longitud, cantidad y SHA-256. El usuario selecciona un ID del catálogo configurado, no una URL arbitraria. Los IDs curados se limitan a words3 hasta words8. [C4]

La intersección es reproducible, pero no garantiza validez lingüística perfecta. Puede excluir flexiones, guiones o apóstrofes y conservar términos raros. Los avisos CMUdict y SCOWL se preservan en data/curated/licenses; no se atribuye una única licencia propia a todas las fuentes.

4 Construcción y algoritmos

4.1 Patrones

Cada posición se sustituye por un asterisco. Cat produce *at, c*t y ca*. Las palabras que comparten patrón se agrupan y conectan: cat y bat comparten *at; cat y cot comparten c*t.

Agrupación por un patrón compartido

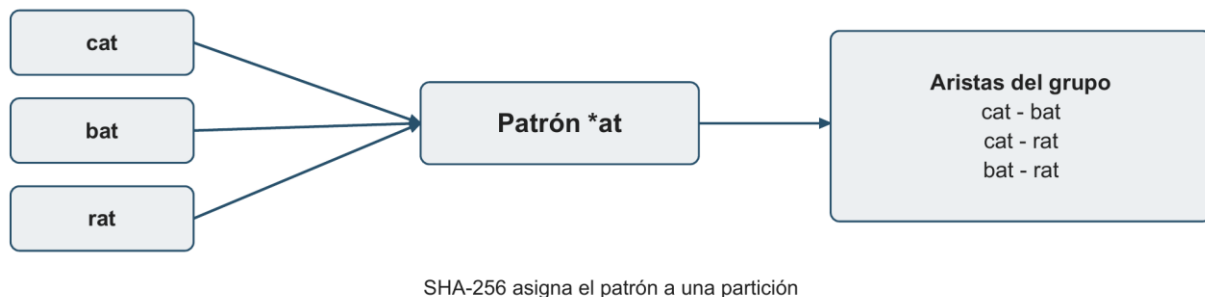


Figura 1. Agrupación por patrones y asignación estable de particiones. Elaboración a partir del motor [C2].

Comparar todas las parejas cuesta aproximadamente $O(N^2L)$, con N palabras y longitud L . Crear L cadenas de longitud L por palabra introduce $O(NL^2)$ en el constructor por patrones, más normalización, ordenación y emisión de aristas. $O(NL)$ omitiría el coste de construir y procesar las cadenas. [C2]

4.2 Reparto y corrección

Cada patrón se asigna mediante el entero SHA-256 módulo el número de particiones. El hash determina su propietario y estabiliza el reparto entre procesos. No decide la semejanza. Patrones diferentes mantienen grupos separados aunque compartan partición.

Dos palabras distintas que difieren en una letra comparten exactamente un patrón. Su asignación única evita perder conexiones entre bloques de palabras. Cada parcial incluye todos los vértices y la reducción une conjuntos de vecinos.

La unión es asociativa, conmutativa e idempotente. Repetir un parcial no duplica aristas. Esta propiedad justifica la reducción, mientras los reintentos necesitan además reservas y comprobación de estado.

Cada partición recorre de nuevo el corpus. El trabajo agregado de P particiones incluye $O(PNL^2)$, además de emitir y unir aristas. El reparto añade lectura, serialización y parciales. Un grupo de un patrón contiene como máximo 26 palabras distintas con alfabeto ASCII.

4.3 Camino mínimo

BFS explora primero vértices a un cambio del origen y después a dos, tres o más. Un mapa de predecesores reconstruye el camino. Como no hay pesos, obtiene el mínimo número de aristas. [C2, R1]

El recorrido convencional cuesta $O(V+E)$. GraphWord ordena vecinos para desempatar reproduciblemente, añadiendo ordenación. Si origen y destino coinciden, devuelve esa palabra; si no hay conexión, devuelve una lista vacía. Una palabra ausente provoca error de validación.

4.4 Caminos simples

Un camino simple no repite vértices. La enumeración usa DFS iterativo y conserva camino y visitados por estado. El número de caminos puede crecer combinatoriamente y se limita por resultados, estados, profundidad y tiempo.

La respuesta informa de complete, stop_reason y explored_states. Complete=false indica exploración parcial. Camino más largo conserva el mejor encontrado y solo acredita optimalidad con complete=true. No equivale al diámetro, definido con distancias mínimas.

El timeout se comprueba entre estados y no interrumpe de forma dura. Max_states cuenta procesados y no fija un techo exacto de memoria de la pila pendiente. [C2]

4.5 Grados componentes y k core

El grado cuenta vecinos; los aislados tienen grado cero. Las componentes reúnen vértices alcanzables. K-core elimina repetidamente vértices cuyo grado interno es inferior a k hasta estabilizar el subgrafo.

Dense-subgraphs devuelve componentes del k-core. Garantiza grado interno mínimo, sin acreditar cliques o densidad global elevada. La literatura presenta algoritmos lineales para cores; este código añade ordenación y conjuntos. [C2, R2]

En words3 curado, el 2-core contiene 593 vértices y una componente; el 3-core, 566 vértices. Conectividad y pertenencia a un core expresan propiedades diferentes.

5 Arquitectura y comunicaciones de la API

5.1 Capas

Create_app recibe repositorios intercambiables. Las rutas validan y responden; el motor calcula; los repositorios guardan; los workers ejecutan; el reconciliador recupera y habilita. El motor no depende de FastAPI o boto3. [C1, C2, C4]

Capa	Responsabilidad	Ubicación
API	Modelos rutas respuestas	graphword/api
Motor	Construcción y consultas	graphword/motor
Almacenamiento	Grafos entradas catálogo estado	graphword/almacenamiento
Trabajos	Reservas ejecución heartbeat	graphword/trabajos

Capa	Responsabilidad	Ubicación
Configuración	Selección de repositorios	graphword/configuracion
Despliegue	Recursos releases operación	scripts/despliegue

5.2 Con quién se comunica

La API recibe HTTP y JSON de Swagger, scripts y aplicaciones. En su proceso llama al motor y a repositorios. En AWS, boto3 opera sobre S3, DynamoDB y SQS. No envía una llamada HTTP directa a los workers para cada tarea.

Variante	Comunicación
En memoria	Motor y repositorios del proceso
Local	Motor y SQLite compartido
Privada EC2	Motor y S3 DynamoDB SQS
Pública Lambda	Mangum FastAPI y servicios compartidos

Lambda ejecuta la aplicación y accede directamente a esos servicios. Lambda no reenvía solicitudes a EC2. Los workers completan trabajos asíncronos; consultas y construcciones síncronas se procesan en la API. [C1, C4, R3]

Administración y despliegue utilizan además CloudFormation, EC2, STS, IAM, SSM, Lambda y CloudWatch. El controlador CD consulta GitHub. Estas comunicaciones pertenecen a operación, no al recorrido ordinario de una consulta.

5.3 Local

API y worker comparten var/graphword.db. SQLite utiliza WAL, espera ante bloqueos y transacciones. La ruta explícita tiene prioridad sobre GRAPHWORD_DB_PATH; ambos procesos deben resolver el mismo archivo. [C4, R4]

La variante en memoria pierde estado al terminar. La local conserva datos sin servicios externos, pero no conecta un catálogo: las rutas de catálogo y construcción desde catálogo devuelven 503.

6 Contrato de la API

6.1 Rutas

FastAPI aporta validación, OpenAPI y Swagger. Las operaciones diferencian resultados inmediatos de trabajos aceptados. [C1, R5]

Método	Ruta	Función
GET	/health	Estado básico y adaptador
GET	/v1/dictionaries	Catálogo configurado
POST	/v1/graphs	Construcción síncrona

Método	Ruta	Función
GET	/v1/graphs/{graph_id}	Resumen
POST	/v1/jobs/graph-builds	Construcción completa asíncrona
POST	/v1/jobs/partitioned-builds	Partición y reducción
POST	/v1/jobs/dictionary-builds	Construcción desde catálogo
GET	/v1/jobs/{job_id}	Estado resultado y error

Las consultas usan el prefijo /v1/graphs/{graph_id}. Bajo /queries hay POST a shortest-path, all-simple-paths y longest-simple-path. GET /nodes filtra grado y GET /dense-subgraphs filtra grado interno mínimo. No se ofrecen borrado, cancelación o permisos por usuario.

6.2 Validación y códigos

Una construcción admite de 1 a 100.000 elementos y de 1 a 64 particiones. Los caminos admiten hasta un millón de estados y 30 segundos opcionales. Todos los caminos llega a 10.000 resultados. Estos límites no garantizan completar cualquier entrada.

HTTP	Significado
201	Grafo construido y guardado
202	Trabajo registrado
404	Recurso inexistente
409	Clave repetida con contenido distinto
422	Entrada inválida o palabra ausente
503	Capacidad no disponible

202 no implica un grafo final: se consulta job_id hasta SUCCEEDED o FAILED y se obtiene graph_id del resultado. Health responde de forma básica y no comprueba todas las dependencias AWS.

6.3 Tres modalidades

POST /v1/graphs calcula y une secuencialmente. Partitions no crea paralelismo. POST /v1/jobs/graph-builds registra un único GRAPH_BUILD que un worker resuelve completo.

POST /v1/jobs/partitioned-builds crea BUILD_PARTITION por partición y REDUCE_GRAPH dependiente de todos los hijos. Permite reparto entre workers. La ruta de catálogo carga y valida el diccionario y utiliza ese mecanismo. [C1, C4, C5]

7 Arquitectura AWS

7.1 Recursos

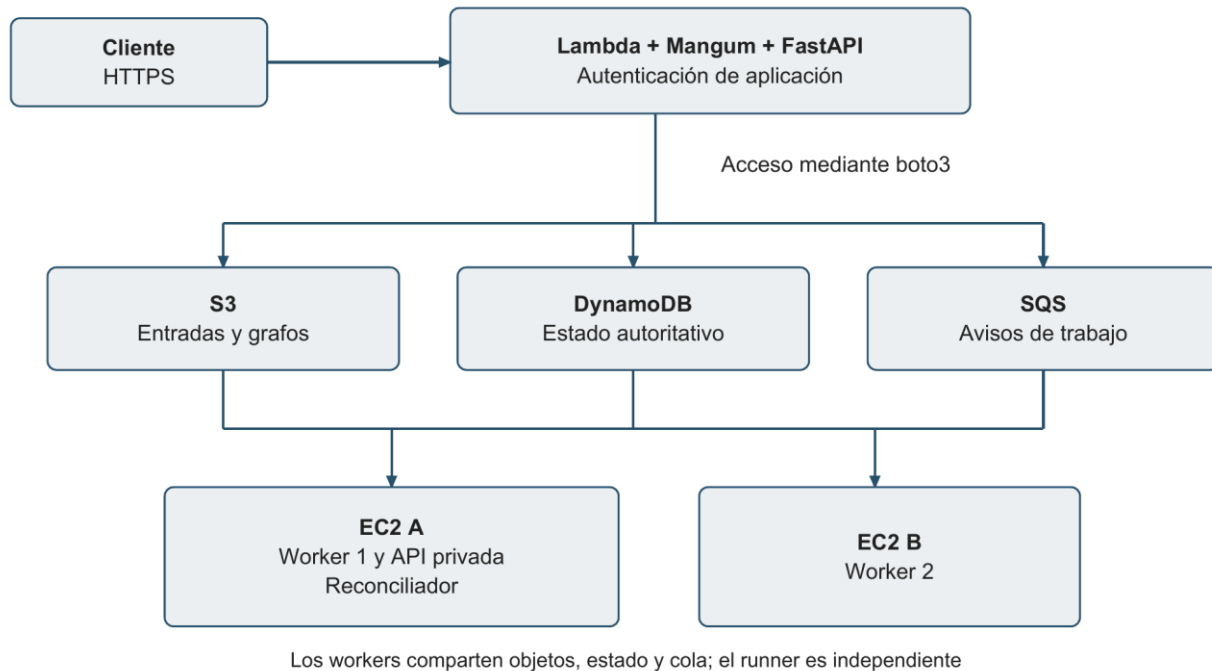


Figura 2. Componentes implementados y repositorios compartidos. El runner de despliegue es independiente [C1, C4, C6].

Recurso	Función
Lambda y Mangum	Entrada HTTPS FastAPI
Dos EC2 t3.micro	Workers
S3	Diccionarios entradas parciales grafos
DynamoDB	Estado dependencias reservas
SQS y DLQ	Avisos y mensajes problemáticos
CloudFormation	Recursos por plantillas
SSM	Administración y acceso privado

El nodo EC2 de API también ejecuta worker y reconciliador. El segundo ejecuta otro worker. El runner GitHub Actions es una tercera instancia y no cuenta como worker del benchmark. [C6]

7.2 Datos y coordinación

S3 guarda entradas y grafos JSON con UUID. Los vecinos se ordenan para serializar. DynamoDB conserva referencias, tipo, dependencias, intentos, revisión, reserva y resultado.

SQS transporta el ID. DynamoDB es autoritativo: recibir un mensaje no basta para ejecutar o confirmar. El reconciliador reconstruye avisos de pendientes, evitando depender exclusivamente de un envío a la cola. [C4, C5]

7.3 Configuración

S3 bloquea acceso público, usa AES256 y conserva datos al eliminar o reemplazar pila. DynamoDB utiliza job_id, cifrado y pago por solicitud. SQS tiene visibilidad de 300 segundos, espera de 10 y redrive tras cinco recepciones; la DLQ retiene 14 días.

EC2 utiliza discos gp3 cifrados, IMDSv2 y LabInstanceProfile. El grupo no declara entrada SSH o puerto 8000. Uvicorn escucha en 127.0.0.1 y SSM facilita administración y túnel. Systemd añade reinicio y restricciones. [C6]

Se reutiliza el rol del laboratorio sin una política IAM propia de privilegio mínimo. Las plantillas no declaran balanceador o autoscaling. ECS y ALB corresponden a propuestas históricas.

8 Coordinación y recuperación

8.1 Ciclo de construcción

La API guarda la entrada en S3, registra padre e hijos y devuelve al cliente el ID del padre. Cada worker recibe aviso, consulta estado y pide reserva. Si la obtiene, calcula, guarda el parcial y confirma. Cuando todos los hijos terminan, el reductor reúne parciales y publica el grafo. [C4, C5]

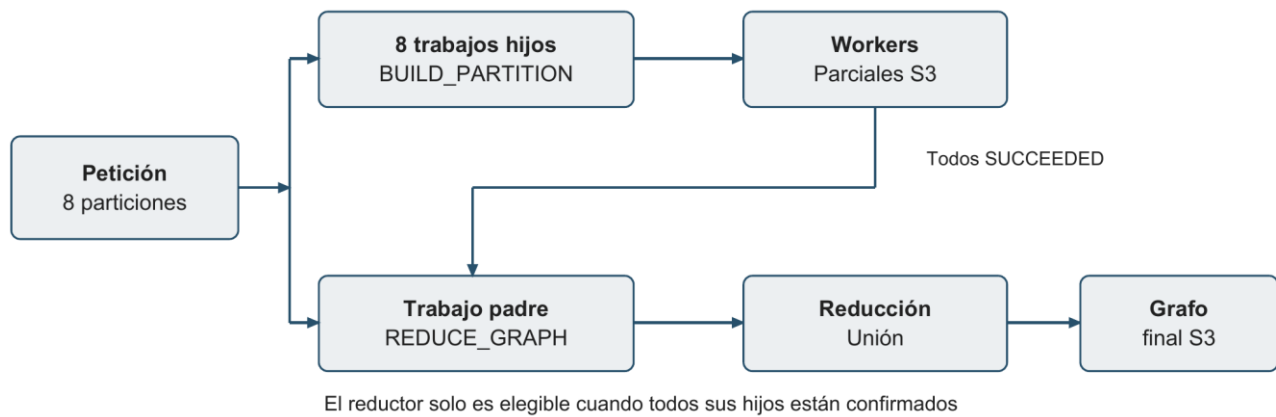


Figura 3. Dependencias de una construcción con ocho particiones. La reducción espera la confirmación de todos los hijos [C4, C5].

Los estados son PENDING, RUNNING, SUCCEEDED y FAILED. El padre puede estar registrado y aún no ser ejecutable. Un hijo con fallo definitivo provoca fallo del reductor.

8.2 Reservas y heartbeat

La reserva contiene worker, token, intento y caducidad. Confirmar exige identidad, token, estado y vigencia. Las escrituras condicionales evitan que un intento antiguo sobrescriba resultados.

Heartbeat renueva cada tercio aproximado de la reserva y también visibilidad SQS. Si falla, impide publicar éxito, sin cancelar forzosamente el cálculo en curso.

El reconciliador recupera reservas y avisa tareas listas cada 30 segundos en la configuración habitual. El barrido completo añade coste y espera.

8.3 Reintentos y mensajes

SQS entrega al menos una vez y puede repetir mensajes. [R6] GraphWord acepta intentos mediante estado y reserva y reconoce avisos atrasados de trabajos ya confirmados.

Se admiten normalmente tres intentos. Recuperables vuelven a PENDING mientras queden; agotarlos provoca FAILED. Las recepciones SQS y los intentos son contadores distintos; la DLQ no sustituye DynamoDB.

No se garantiza ejecución física exactamente una vez. Una reserva caducada puede causar cálculo redundante, aunque solo el intento vigente confirma. No hay transacción conjunta S3, DynamoDB y SQS.

8.4 Idempotencia

Idempotency-Key y SHA-256 del contenido canónico identifican un envío. La misma clave y contenido devuelve el trabajo existente; otro contenido produce 409. Sin clave se crea una operación diferente.

No hay separación de claves por usuario o expiración declarada. La idempotencia del envío evita duplicación de peticiones; la unión idempotente evita duplicar aristas. [C5]

9 Seguridad y operación

9.1 Acceso público

Lambda usa Python 3.11, 512 MiB, timeout 35 segundos y concurrencia reservada dos. La Function URL tiene AuthType NONE e invocación pública. La contraseña se comprueba dentro de la aplicación. [C6, R7]

Se deriva con PBKDF2-HMAC-SHA256, sal y 200.000 iteraciones; se compara con compare_digest. Basic Auth sirve a programas y el formulario al navegador. Cookie HMAC-SHA256 dura dos horas, con Secure, HttpOnly y SameSite Strict.

Swagger, OpenAPI, health y datos quedan protegidos. La página incorpora controles de caché, marcos y contenido. Deploy exige contraseña de al menos 16 caracteres y conserva sal y derivado. La contraseña se entrega por separado.

9.2 Límites

AuthType NONE permite llegar a Lambda antes de autenticar. Rechazar protege datos, pero consume recursos. La concurrencia no limita intentos por usuario. [C6, R7]

La credencial es compartida, sin permisos por grafo, revocación individual o logout. El derivado también firma sesiones. Para uso continuo se propone separar secretos, identidad individual y control de intentos.

La aplicación local y la EC2 no utilizan esa envoltura pública; dependen de red y acceso IAM y SSM.

9.3 Pausa y conservación

Pausar fija concurrencia Lambda cero y verifica; conserva URL. Workers y runner se detienen separadamente. Parar cómputo no elimina bucket, tabla u objetos.

No hay expiración general de trabajos, claves o grafos ni recogida de huérfanos. La regla S3 elimina multipart incompletos, no resultados antiguos. Conservar facilita reanudar y requiere gestionar retención. [C6]

10 Demostración funcional

10.1 Acceso

La documentación conserva [Swagger HTTPS de GraphWord](#). Requiere contraseña. La construcción síncrona necesita API y repositorios; la distribuida, workers y reconciliador.

La URL documentada no certifica que Lambda esté reanudada al leer. [E3]

10.2 Ejemplo pequeño

POST /v1/graphs admite:

```
{
  "words": ["cat", "bat", "bad", "dad", "mat", "rat", "zzz"],
  "partitions": 2
}
```

Se esperan HTTP 201, siete vértices, ocho aristas y dos componentes. Zzz es aislado. Se conserva graph_id para consultar.

Consulta	Resultado esperado
Mínimo cat a dad	cat, bat, bad, dad
Todos los caminos cat a dad	Cinco caminos
Más largo cat a dad	cat, mat, rat, bat, bad, dad
Grado cero	zzz
Grado cuatro	bat
k-core mínimo tres	bat, cat, mat, rat

Para este caso se pueden fijar 10.000 estados, profundidad 10, cinco segundos y hasta 100 caminos. Debe comprobarse complete. Los resultados corresponden a esta lista.

10.3 Catálogo

Tras GET /v1/dictionaries, se envía a POST /v1/jobs/dictionary-builds:

```
{"dictionary_id": "words3", "partitions": 8}
```

202 entrega el ID. Se consulta /v1/jobs/{job_id} hasta SUCCEEDED y se recupera result.graph_id. Words3 curado produce 628 vértices, 3.815 aristas y 12 componentes.

Su mínimo cat a dad es cat, cad, dad. El archivo histórico data/words3.txt produce cat, bat, bad, dad. La presencia de cad explica el cambio.

10.4 Evidencia

HTTPS 37440936427, revisión 2d30e92, verificó autenticación, formulario, cookie, OpenAPI, Swagger, catálogo, grafo síncrono y camino mínimo. No ejercitó workers asíncronos.

EC2 37436488675, revisión 9ca75f0, verificó construcción distribuida, dos workers, igualdad, idempotencia y words3 desde S3. Son alcances y revisiones distintos. Los ejemplos son instrucciones reproducibles, no nuevas llamadas AWS de esta redacción. [E2, E3]

10.5 Capturas de la API

Las siguientes capturas muestran la documentación interactiva Swagger UI de la aplicación GraphWord API 0.7.0 y una solicitud ejecutada desde esa interfaz. Se obtuvieron el 8 de octubre de 2026 en una ejecución local de graphword.api.local_app, con SQLite y una base de datos aislada para la demostración. Son evidencia del funcionamiento local de la API y no capturas del despliegue AWS. La fecha de portada se conserva conforme a los datos de entrega. [E7]

GraphWord API 0.7.0 OAS 3.1

/openapi.json

Word graphs with interchangeable local and AWS storage adapters. Partitioned jobs are processed by independent workers.

operations		^
GET	/health Health	▼
dictionaries		^
GET	/v1/dictionaries List Dictionaries	▼
jobs		▼
graphs		^
POST	/v1/graphs Create Graph	▼
GET	/v1/graphs/{graph_id} Get Graph	▼
queries		^
POST	/v1/graphs/{graph_id}/queries/shortest-path Query Shortest Path	▼
POST	/v1/graphs/{graph_id}/queries/all-simple-paths Query All Simple Paths	▼
POST	/v1/graphs/{graph_id}/queries/longest-simple-path Query Longest Simple Path	📄 ▼
GET	/v1/graphs/{graph_id}/nodes Query Nodes By Degree	▼
GET	/v1/graphs/{graph_id}/dense-subgraphs Query Dense Subgraphs	▼
Schemas		▼

Figura 4. Swagger UI de GraphWord API 0.7.0. Captura local del 8 de octubre de 2026; el grupo jobs está contraído [E7].

En POST /v1/graphs se enviaron las palabras cat, bat, bad, dad, mat, rat y zzz, con dos particiones. El servidor respondió HTTP 201 Created y devolvió 7 nodos, 8 aristas, 2 componentes, zzz como nodo aislado y bat como nodo de grado máximo, con grado 4. El resultado coincide con el ejemplo pequeño de la sección 10.2. Esta operación demuestra la creación síncrona; no mide el rendimiento de workers EC2.

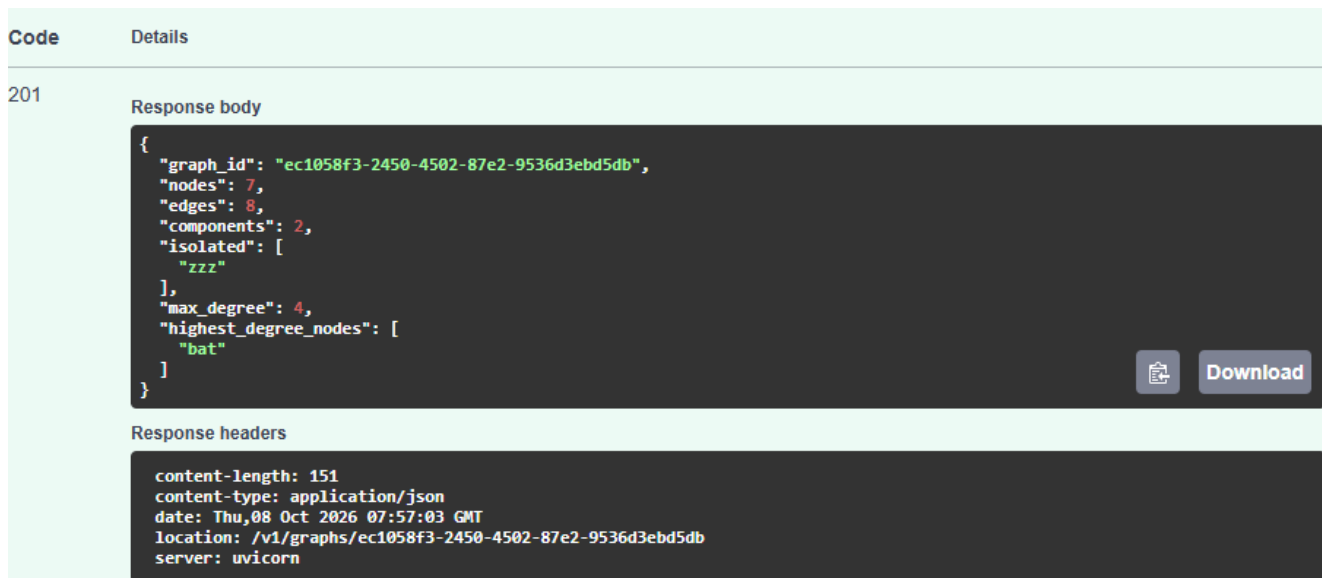


Figura 5. Respuesta real HTTP 201 a POST /v1/graphs. Captura local del 8 de octubre de 2026 [E7].

11 Pruebas y validación

11.1 Estrategia

Unittest verifica motor, API, persistencia, coordinación y scripts. Moto simula S3, DynamoDB y SQS; deploy utiliza mocks. Smoke AWS añade integración real en ejecuciones separadas. [C7]

El oráculo compara parejas. Se verifican 20 diccionarios reproducibles y 1, 2, 7 y 31 particiones: 80 combinaciones. También estabilidad entre intérpretes, unión, caminos, truncamiento y k-core.

11.2 CI histórico

La ejecución 37440936429 del 6 de octubre de 2026, revisión 2d30e92, conserva:

Entorno	Casos	Resultado	Tiempo unittest
Ubuntu 22.04 y Python 3.11	122	OK	11,518 s
Ubuntu 22.04 y Python 3.12	122	OK	12,843 s

Son los mismos 122 casos en dos intérpretes, no 244 diferentes. Los tiempos son de tests y los logs no indican cobertura completa de líneas o ramas. [E1]

11.3 Revisión local del 7 de octubre

Proyecto	Python	Casos	Resultado	Tiempo
GraphWord-work	3.12.14	122	OK	27,164 s
GraphWord-python	3.12.10	12	OK	0,931 s
GraphWord-CD	3.12.10	16	OK	0,027 s

Se utilizan entornos locales, bytecode desactivado y PYTHONPATH separado, sin AWS. Mangum emite advertencia asyncio deprecada sin fallo. No se repite localmente Python 3.11. [E6]

Área actual	Casos
Motor	18
API	10
AWS simulado	13
CI deploy y workflow	13
Plantillas y paquete	4
Diccionarios	9
Robustez e idempotencia	8
Jobs	5
EC2 y runner con mocks	16
Construcciones particionadas	5
API pública Lambda	16
SQLite	5
Total	122

11.4 Interpretación

Igualdad de adyacencia completa es más fuerte que nodos y aristas: grafos distintos pueden compartir cantidades. El smoke también verifica replay y conflicto.

Moto no reproduce cuotas, latencia, permisos reales y todos los fallos. Aprobar tests no acredita una campaña de caída de todas las máquinas o disponibilidad formal. CI histórico y repetición local son evidencias complementarias de revisiones y entornos distintos.

12 Integración y despliegue continuos

12.1 CI

GitHub Actions instala, ejecuta Python 3.11 y 3.12 y conserva logs. Pyproject fija directas, pero no el cierre completo de transitivas.

Declara FastAPI 0.141.1, Uvicorn 0.52.4, boto3 1.43.107, Moto 5.2.3, Mangum 0.22.0 y httpx2 2.12.0 para tests. Son versiones del archivo, no versiones más recientes. [C7]

12.2 CD de aplicación

Exige tests correctos, rama y evento autorizados, cuenta y ENABLE_LAB_CD. El runner EC2 graphword-lab usa rol de instancia. Descarga la revisión, publica release, aplica CloudFormation, ejecuta smoke SSM y verifica parada.

El ZIP usa lista permitida, excluye credenciales, entornos y Git, fija fechas y permisos y tiene SHA-256. Aporta trazabilidad, aunque AML dinámica y transitivas limitan reproducción binaria. [C6, C7]

Un workflow público permite deploy o pause; ENABLE_PUBLIC_API lo integra. Finally solicita parar EC2, pero terminación forzada o fallo temprano puede impedir limpieza.

12.3 GraphWord CD

El controlador verifica repositorio, rama, evento, workflow, revisión y ambos jobs Python. Consulta GitHub con token temporal y registra Deployments. Cada SHA se intenta una vez automáticamente; force es manual.

Revisa cada 15 minutos cuando se habilita. El runner es infraestructura administrada por el proyecto y hay que controlar quién ejecuta código con su acceso AWS. [C7, R8]

12.4 Trazabilidad

Ejecución	Revisión	Alcance
CI 37440936429	2d30e92	122 por intérprete; deploy skipped
HTTPS 37440936427	2d30e92	Lambda y smoke síncrono
EC2 37436488675	9ca75f0	103 tests, smoke y parada

Skipped no acredita otro deploy. El smoke de 9ca75f0 no se atribuye íntegro a 2d30e92. Los enlaces necesitan permisos si el repositorio está restringido. [E1-E3]

13 Diseño experimental

13.1 Condiciones

Se estudia si dos workers reducen tiempo manteniendo corpus y particiones. AWS usa t3.micro, us-east-1, ocho particiones, palabras de tres a seis letras y tres repeticiones.

Cuatro diccionarios por dos modos por tres repeticiones produce 24 construcciones. Se alterna el orden y se compara cada grafo con el oráculo. [E4]

13.2 Tiempo medido

Empieza con HTTP a la API privada EC2 y termina al observar SUCCEEDED. Incluye S3, DynamoDB, SQS, workers, reducción y sondeo. Excluye arranque, parada, despacho SSM, referencia y descarga posterior para igualdad.

El diccionario inicial se lee en EC2; entradas y grafos utilizan S3. No equivale a petición de catálogo mediante Lambda. La reconciliación experimental ocurre cada 0,5 segundos y después se restaura; la habitual usa 30 segundos.

13.3 Métricas

Se usa mediana. La aceleración divide tiempo de un worker entre dos. La reducción porcentual es uno menos el cociente inverso, multiplicado por cien.

Mide latencia de una construcción, no throughput o escalado lineal. Tres valores por combinación ofrecen evidencia inicial con incertidumbre.

14 Benchmark y discusión

14.1 AWS

Diccionario	Vértices	Aristas	Un worker s	Dos workers s	Aceleración
words3	628	3.815	1,966	1,417	1,39
words4	1.889	7.153	1,987	1,424	1,40
words5	2.677	3.171	2,158	1,481	1,46
words6	3.676	2.332	2,084	1,498	1,39

Tiempo de construcción en AWS

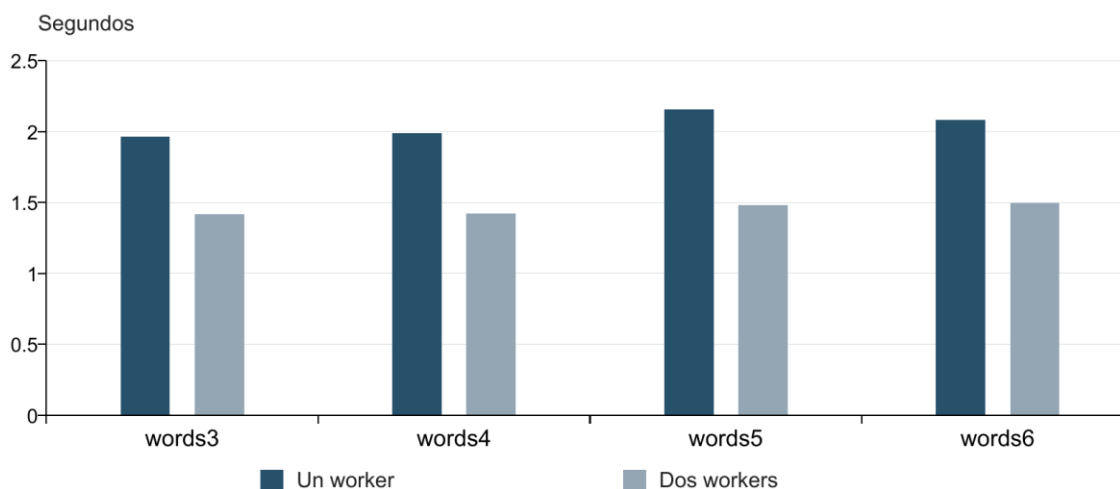


Figura 6. Medianas de tres repeticiones por configuración. Ocho particiones y reconciliación cada 0,5 s [E4].

Dos workers reducen medianas aproximadamente entre 28 % y 31 %. Los 24 registros coinciden con el oráculo, workers observados y configurados concuerdan y los ocho parciales tienen un intento en cada construcción. Las medianas recalculadas coinciden con el resumen. [E4, E6]

Diccionario	Rango un worker s	Rango dos workers s
words3	1,918 a 1,978	1,401 a 1,452
words4	1,983 a 2,077	1,406 a 1,513
words5	2,052 a 2,599	1,446 a 1,554
words6	2,078 a 2,704	1,462 a 2,050

Los rangos muestran variación y no son intervalos de confianza. Aceleración menor que dos es compatible con coordinación, lecturas, almacenamiento y reducción. Es interpretación arquitectónica; no se ha perfilado cada coste en CPU o red.

14.2 Local

Hay 60 construcciones: seis longitudes, dos modos y cinco repeticiones. ProcessPoolExecutor usa Windows 10, ocho CPU lógicas, Python 3.12.14 y ocho particiones. Todas coinciden con el oráculo. [E5]

Diccionario	Un proceso s	Dos procesos s	Aceleración
words3	0,298	0,282	1,05
words4	0,454	0,379	1,20
words5	0,575	0,435	1,32
words6	0,748	0,549	1,36
words7	0,878	0,611	1,44
words8	0,898	0,637	1,41

Incluye creación de procesos, cálculo, IPC, unión y cierre del pool. Excluye lectura, referencia e igualdad, sin AWS o Moto.

Local y AWS no son directamente comparables por hardware y fronteras de medida. Ambos muestran mejora bajo sus condiciones.

14.3 Validez

Tres repeticiones AWS no estiman percentiles o tráfico variable. No se mide coste, concurrencia, créditos CPU, caídas o corpus mucho mayores.

Particiones fijas no determinan su óptimo o un tercer worker. La continuación debe variar corpus, densidad, particiones y workers separadamente, ampliar repeticiones y dispersión y mantener acceso y reconciliación constantes.

15 Limitaciones y mejoras

15.1 Escala

Reducción y consultas cargan el grafo completo; cada partición relee entrada. El límite de elementos no garantiza completar todas las longitudes y tamaños.

El reconciliador escanea DynamoDB y comparte EC2 con API. No hay redundancia de ese nodo, balanceador o autoscaling. Reparto y reintentos no acreditan disponibilidad permanente.

15.2 Dominio propio DNS y balanceo de carga

La demostración no incorpora un dominio propio ni una zona DNS gestionada para el proyecto. El acceso público utiliza el nombre asignado por AWS a la Lambda Function URL. Esta dirección ya

permite HTTPS, pero el proyecto no ofrece una dirección personalizada independiente del recurso de entrada. Esta es una limitación del alcance del despliegue presentado. [C6, E3]

La API privada en EC2 se ejecuta en un único nodo, que también alberga el reconciliador. No se ha desplegado un Application Load Balancer ni varias réplicas de esa API. Si ese nodo deja de estar disponible, su servicio HTTP y la reconciliación quedan afectados. La entrada pública mediante Lambda es una aplicación separada que accede directamente a los servicios compartidos; no actúa como proxy de la API EC2. [C1, C6]

Los dos workers y SQS reparten trabajos de construcción, pero no balancean las peticiones HTTP entre réplicas de la API. Añadir únicamente un registro DNS tampoco elimina la dependencia de un único servidor. Por ello, el benchmark acredita reparto del cálculo, sin demostrar alta disponibilidad de la capa HTTP.

Estas carencias se presentan como limitaciones y trabajo futuro. Las evidencias revisadas no acreditan una denegación específica de AWS Academy para crear DNS o balanceadores; no se atribuye a la plataforma una prohibición no demostrada. La ampliación requerirá comprobar los permisos disponibles y su coste.

15.3 Consistencia y observabilidad

Guardar S3 antes de registrar o confirmar puede dejar huérfanos. Retener indefinidamente trabajos y claves aumenta almacenamiento y barridos.

EC2 registra en journal; órdenes SSM pueden enviar CloudWatch. No hay centralización continua de todos los procesos o alarmas declaradas. Health no prueba preparación de repositorios.

15.4 Mantenimiento

Documentos con pendientes superados deben fecharse y referenciar su revisión. Scripts/build_corpus.py debe actualizarse a scripts/datos/build_corpus.py.

GraphWord-CD describe runner exclusivo privado y GraphWord-work también usa graphword-lab. El código no establece su conexión actual; documentación y configuración deben reconciliarse.

15.5 Trabajo futuro

Prioridad	Mejora	Verificación
Alta	Smoke Lambda y EC2 de igual revisión	Trazabilidad completa
Alta	Identidad y permisos individuales	Revocación comprobada
Alta	Entrada limitada y readiness	Rechazo temprano
Media	Dominio propio y balanceo de la API EC2	HTTPS y continuidad ante caída de una réplica
Media	Índices y outbox	Menos escaneos y avisos recuperables
Media	Retención y huérfanos	Limpieza de datos no activos
Media	Logs métricas y alarmas	Detección de fallos
Posterior	Nuevas cargas y configuraciones	Curvas de rendimiento

Como trabajo futuro para la variante EC2, se propone registrar un dominio propio, configurar su DNS y dirigirlo a un Application Load Balancer con al menos dos réplicas de la API. Route 53 permite crear un registro alias hacia el balanceador; un listener HTTPS puede utilizar un certificado de AWS Certificate Manager. El balanceador comprobaría el estado de las réplicas y distribuiría las solicitudes entre las disponibles. Esta propuesta amplía la variante EC2 y no describe la entrada Lambda actualmente documentada. [R9, R10]

Las réplicas deberán usar los repositorios compartidos de AWS y no depender de estado local. Antes de incorporar una réplica al balanceador se comprobará su preparación para acceder a esos repositorios. El reconciliador necesita un diseño de recuperación propio: añadir réplicas HTTP no resuelve por sí solo su dependencia del nodo actual.

La validación de esta ampliación incluirá resolución del dominio, HTTPS válido, solicitudes concurrentes y retirada controlada de una réplica para comprobar continuidad y medir errores y latencia. Se mantendrán la igualdad del grafo, la idempotencia y el rechazo de confirmaciones caducadas. Son propuestas pendientes de implementación.

16 Conclusiones

GraphWord implementa un modelo preciso y un constructor por patrones cuyo reparto conserva conexiones. La separación de motor, API, almacenamiento y trabajos permite reutilizar y contrastar cálculo.

AWS utiliza Lambda para entrada, EC2 para workers, S3 para objetos, DynamoDB para coordinación y SQS para avisos. Reservas, heartbeat e idempotencia atienden fallos concretos. Las consultas siguen locales a la API y confirmación válida no equivale a ejecución física exactamente una vez.

La revisión local reproduce 122 casos. CI y despliegues documentan revisiones y alcances distintos. Las aceleraciones AWS de 1,39 a 1,46 respaldan el reparto en las cuatro cargas, sin demostrar escalado lineal o tráfico concurrente.

El proyecto conecta algoritmos, persistencia, coordinación, automatización y evaluación. Su continuación puede medirse con integración de una revisión única, campañas más amplias y reducción de cuellos de memoria y reconciliación.

Referencias

Código

Las rutas se interpretan desde GraphWord-python-etapa-1.

[C1] GraphWord-work/graphword/api. Rutas y modelos: routes.py, líneas 46 a 114 y 157 a 523.

Adaptadores: local_app.py, aws_app.py, lambda_app.py. Seguridad: public_security.py y browser_sessions.py. Runtime: graphword/configuracion.

[C2] GraphWord-work/graphword/motor/graph.py. Normalización y construcción, 33 a 91; caminos, 94 a 259; componentes y cores, 262 a 328; resumen, 331 a 357. Antecedente en GraphWord-python/graphword/graph.py.

[C3] GraphWord-work/data/curated/manifest.json, README.md, diccionarios y licenses. Script: scripts/datos/build_corpus.py. LibreOffice 32b006a2c22a4ac7e8ed3f03346f7b3d85a970a4; CMUdict 74790861f652b15e4ac49015a90074ad62a27690.

[C4] GraphWord-work/graphword/almacenamiento. Catálogo en aws_storage.py, 38 a 117; objetos, 140 a 176; coordinación, 249 a 512. SQLite en sqlite_storage.py.

[C5] GraphWord-work/graphword/trabajos. worker.py, 24 a 68; submissions.py, 12 a 36; heartbeat.py, jobs.py, worker_cli.py y aws_worker.py.

[C6] GraphWord-work/infra/storage.json. Scripts/despliegue/deploy_aws.py, bootstrap_aws.py, public_api.py, ci_deploy.py; scripts/operaciones. Lambda: public_api.py, 31 a 75.

[C7] GraphWord-work/pyproject.toml, README.md, tests y .github/workflows. GraphWord-CD/scripts/control.py, verify_ec2.py y .github/workflows/deploy.yml. Documentación histórica y de entrega.

Registros

[E1] informe/evidencias/ci-python-3.11/unittest.txt y informe/evidencias/ci-python-3.12/unittest.txt. [CI 37440936429](#), 2d30e92.

[E2] informe/evidencias/resumen-smoke-ec2.json. [EC2 37436488675](#), 9ca75f0.

[E3] informe/evidencias/aws-public-api.json y GraphWord-work/docs/entrega/acceso-publico-https.md. [HTTPS 37440936427](#), 2d30e92.

[E4] informe/evidencias/benchmark-aws.json; GraphWord-work/scripts/rendimiento/benchmark_aws.py y GraphWord-work/scripts/rendimiento/benchmark_remote.py. 24 construcciones, tres repeticiones.

[E5] informe/evidencias/benchmark-local.json; GraphWord-work/scripts/rendimiento/benchmark_local.py. 60 construcciones, cinco repeticiones.

[E6] informe/evidencias/validacion-local-2026-10-07.json. Pruebas y comprobaciones locales de esta revisión.

[E7] informe/evidencias/captura-api-local-2026-10-08.json y capturas de Swagger en informe/output/playwright. Solicitud POST /v1/graphs ejecutada desde el navegador el 8 de octubre de 2026; HTTP 201, almacenamiento local SQLite.